

Esame OS Internals (Cabodi)

Data: 02/09/2021

QUESTIONS

SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Memoria Virtuale, Second Chance e Codice C)

Contesto: Sia dato il frammento di programma rappresentato:

```
#define N (128*1024)
int i, j;
int A[1024], B[N];

for (j=0; j<N; j++) {
    B[j] = 0;
}

for (i=0; i<1024; i++) {
    j = A[i];
    if (j>=0 && j<N)
        B[j]++;
}
```

Il codice macchina generato da tali istruzioni viene eseguito in un sistema con memoria virtuale gestita mediante paginazione, con pagine di 2 KByte, utilizzando come politica di sostituzione pagine la SECOND CHANCE. Si sa che un `int` ha dimensione 32 bit e che le istruzioni in codice macchina, corrispondenti al frammento di programma visualizzato, sono contenute in una sola pagina. Si supponga che A e B siano allocati ad indirizzi logici contigui (prima A, poi B), a partire dall'indirizzo logico 0x74240A00.

Si risponda alle seguenti domande:

-) Quante pagine (e frame) sono necessarie per contenere i vettori A e B?
-) Ipotizzando che le variabili i, j siano allocate in registri della CPU, quanti accessi in memoria (in lettura e scrittura) fa il programma proposto, per accedere a dati (non vanno conteggiati gli accessi a istruzioni)? (Siccome il costrutto `if (j>=0 && j<N)` condiziona un'istruzione, si calcolino sia il caso minimo che il caso massimo).
-) Calcolare il numero di page fault generati dal programma proposto, supponendo che siano allocati per esso 10 frame, di cui uno utilizzato (già all'inizio dell'esecuzione) per le istruzioni. (Considerare nuovamente caso minimo e massimo, e motivare le risposte).

Domanda 2 (Memorie di Massa e Dischi SSD)

Contesto: Sia dato un disco di tipo "solid state" (SSD). Una delle tecniche usate per prolungare la vita di un dispositivo SSD è il cosiddetto "wear leveling", che consiste nel mantenere una tabella (mappa) di corrispondenza tra indirizzi logici e fisici per i blocchi su disco. Implementata in varie modalità, tale tecnica cerca di "spostare" un dato blocco logico da un blocco fisico (pagina, nella terminologia SSD) a un altro in corrispondenza a un'operazione di "ri-scrittura".

Si risponda alle seguenti domande:

-) Perché è opportuna questa operazione?
 -) Da cosa nasce il possibile vantaggio, da un numero minore complessivo di scritture su disco, oppure da qualcos'altro?
 -) E' possibile che un disco SSD continui a funzionare correttamente una volta che una o più pagine (blocchi) fisiche siano marcate come inutilizzabili (in quanto hanno raggiunto il numero massimo di riscritture)? (Se sì, spiegare come; se no, motivare).
-

Domanda 3 (I/O Sincrono, Asincrono e Bloccante)

Contesto: Si considerino i due tipi di sincronizzazione, relativi a operazioni di I/O: sincrono e asincrono.

Si risponda alle seguenti domande:

-) Si spieghino le principali differenze tra gli I/O dei due tipi. Si considerino poi I/O bloccante e non bloccante: si tratta di sinonimi di asincrono e sincrono? Ci sono differenze (tra bloccante/non-bloccante e sincrono/asincrono)? (se no, perché; se sì, quali?).
-) Un I/O sincrono può essere effettuato in polling oppure è necessario effettuarlo in interrupt? (Si risponda alla stessa domanda per il caso dell'I/O asincrono).
-) In quale modo può un processo beneficiare di un I/O asincrono? E' possibile, nel caso di I/O asincrono, scrivere un programma con istruzioni, successive all'I/O, che dipendano dai dati coinvolti (ad es. letti) durante l'I/O?

SECTION 2: OS161

Domanda 4 (System Call `sys_read` e Memoria)

Contesto: Sia dato un sistema OS161. Si consideri la funzione `sys_read`, utilizzata per realizzare la system call `read`. Se ne propone una realizzazione, limitata al caso di standard input.

```
int sys_read(int fd, userptr_t buf, int size) {
    if (fd == STDIN_FILENO) {
        kgets ((char *)buf, size);
        return size;
    } else {
        kprintf("sys_read implemented only on stdio\n");
        return -1;
    }
}
```

Si risponda alle seguenti domande:

-) E' possibile che la funzione sia richiamata direttamente da un processo user (ad esempio mediante una chiamata interna a `palin.c`, `sort.c`, `forktest.c`)? (Motivare la risposta).
 -) La funzione è errata, in quanto la chiamata a `kgets`, così come effettuata, rischia di fare un accesso scorretto in memoria. Si spieghi il perché e si proponga una possibile correzione al problema.
 -) Perché la funzione ritorna `size`, qualora non ci sia errore? E' possibile che, in una realizzazione più completa della funzione, il valore ritornato sia maggiore o minore di `size`?
-

Domanda 5 (Sincronizzazione Multi-core e Semafori)

Contesto: Sia dato un sistema operativo OS161.

Si risponda alle seguenti domande:

-) Si spieghi perché, in un contesto multi-core (sono presenti più CPU) non è possibile realizzare la mutua esclusione semplicemente disabilitando e riabilitando l'interrupt.
-) Dato codice (ridotto alle parti essenziali) delle funzioni di semaforo P e V, riportate in seguito:

```
void P(struct semaphore *sem) {
    spinlock_acquire(&sem->sem_lock);
    while (sem->sem_count == 0) {
        wchan_sleep(sem->sem_wchan, &sem->sem_lock);
    }
    sem->sem_count--;
    spinlock_release(&sem->sem_lock);
}

void V(struct semaphore *sem) {
    spinlock_acquire(&sem->sem_lock);
```

```
sem->sem_count++;  
KASSERT(sem->sem_count > 0);  
wchan_wakeone(sem->sem_wchan, &sem->sem_lock);  
spinlock_release(&sem->sem_lock);  
}
```

1. A cosa serve lo spinlock (in entrambe le funzioni)?
 2. Perché la P contiene un ciclo `while` anziché un `if (sem->sem_count == 0)?`
 3. Perché la `wchan_sleep` riceve come parametro lo spinlock?
 4. E' possibile che la chiamata alla `wchan_wakeone` svegli più di un thread in attesa su `wait_channel`?
-) Spiegare brevemente la differenza tra condition variable e `wait_channel`.

ANSWERS

Domanda 1 (Memoria Virtuale, Second Chance e Codice C)

Dati Iniziali:

- `sizeof(int)` = 4 Byte.
- Dimensione pagina = 2 KB = 2048 Byte (0x800 in esadecimale).
- Frame disponibili per i dati = 10 totali - 1 per il codice = 9 frame.
- Dimensione A = 1024×4 = 4096 Byte.
- Dimensione B = $128 \times 1024 \times 4$ = 524.288 Byte.

A) Pagine e frame necessari per i vettori A e B:

I vettori sono allocati contigualmente a partire dall'indirizzo 0x74240A00. Questo indirizzo non è allineato al confine di una pagina da 2048 byte (0x800). Dividendo l'indirizzo per 0x800, si ottiene un resto di 0x200, ovvero un offset di **512 Byte**. Questo significa che l'array A inizia dopo 512 Byte dall'inizio della pagina logica.

$$\text{Dimensione combinata} = 4096 + 524.288 = 528.384 \text{ Byte.}$$

Il numero totale di pagine richieste deve coprire sia l'offset iniziale che la dimensione dei dati:

$$\text{Pagine totali} = \lceil \frac{\text{Offset} + \text{Dimensione}}{\text{Dimensione Pagina}} \rceil = \lceil \frac{512 + 528.384}{2048} \rceil = \lceil 258,25 \rceil = \mathbf{259} \text{ pagine.}$$

In RAM saranno perciò necessari **259 frame** per contenere interamente entrambi i vettori.

B) Accessi totali a memoria (Minimo e Massimo):

Le variabili i e j sono mantenute nei registri della CPU.

- **Primo ciclo for:** Esegue $B[j] = 0;$. Viene effettuata solo 1 scrittura per iterazione.
Accessi = 128×1024 = **131.072**.
- **Secondo ciclo for (Caso Minimo):** La condizione dell'`if` risulta sempre falsa. Viene eseguita solo l'istruzione $j = A[i];$ (1 Lettura per iterazione).
Accessi = 1024×1 = **1024**.
Totale Minimo = $131.072 + 1024$ = **132.096** accessi.
- **Secondo ciclo for (Caso Massimo):** La condizione dell'`if` risulta sempre vera. L'istruzione $B[j]++$ si espande in $B[j] = B[j] + 1$, richiedendo quindi **1 Lettura e 1 Scrittura**. Sommate alla lettura di $A[i]$, si hanno 3 accessi per iterazione.
Accessi = 1024×3 = **3072**.
Totale Massimo = $131.072 + 3072$ = **134.144** accessi.

C) Numero di Page Fault generati (Minimo e Massimo):

Si hanno a disposizione 9 frame fisici per i dati. L'array A, per via dell'offset, "sporca" esattamente 3 pagine. L'array B, trovandosi contiguo ad A, tocca le restanti 257 pagine.

- **Analisi Primo Ciclo:** Si inizializza l'intero array B. Si verificheranno esclusivamente i miss obbligatori per le 257 pagine di B.
Al termine del ciclo, negli 8 frame dati risiederanno le ultime 9 pagine dell'array B (la politica d'uscita riempie i frame a rotazione).
- **Secondo Ciclo (Caso Minimo):** Si accede solo all'array A (che risiede nelle primissime 3 pagine allocate, attualmente assenti in memoria). Si genereranno esattamente 3 Page Fault per ricaricare A.
PF Totali Minimi = 257 (Ciclo 1) + 3 (Ciclo 2) = **260** Page Faults.
- **Secondo Ciclo (Caso Massimo):** Si accede sia ad A che a B. Leggere A genera 3 PF. Ad ogni iterazione si accede in modo pseudo-casuale a B. Nel caso peggiore avversario, ogni $B[j]$ indirizza una pagina non presente nei 9 frame fisici, generando un Page Fault per ogni iterazione (1024 PF per B). Le 3 pagine di A, essendo lette ad *ogni singola iterazione*, avranno sempre il proprio *Reference Bit* settato a 1: di conseguenza, la politica Second Chance le proteggerà e non le sfratterà mai, impedendo ulteriori miss per A.
PF Totali Massimi = 257 (Ciclo 1 per B) + 3 (Ciclo 2 per A) + 1024 (Ciclo 2 per B) = **1284** Page Faults.

Domanda 2 (Memorie di Massa e Dischi SSD)

A) Scopo dell'operazione di Wear Leveling:

L'operazione è fondamentale a causa della natura fisica delle memorie NAND Flash, le quali supportano un numero limitato di cicli di cancellazione/scrittura (*Program/Erase cycles*) prima di degradarsi e guastarsi. Il *wear leveling* (livellamento dell'usura) sfrutta la tabella di mapping logico-fisico (gestita dal controller FTL dell'SSD) per mappare dinamicamente un medesimo blocco logico su blocchi fisici sempre diversi ad ogni sovrascrittura. Questo evita che indirizzi logici soggetti ad altissima frequenza di scrittura (es. tabelle del file system) usurino e distruggano precocemente un singolo blocco fisico, compromettendo l'unità.

B) Origine del vantaggio competitivo:

Il vantaggio **non** nasce da un minor numero di scritture su disco (che, al contrario, possono addirittura aumentare a causa delle operazioni di background del controller per spostare i dati statici, fenomeno noto come *Write Amplification*). Il vantaggio nasce esclusivamente dalla **distribuzione uniforme dell'usura**. Evitando che l'usura si concentri su un sottoinsieme ristretto di blocchi fisici, si massimizza la longevità complessiva e il MTBF (*Mean Time Between Failures*) dell'intero dispositivo.

C) Funzionamento a seguito di blocchi inutilizzabili:

SÌ, è assolutamente possibile che il disco continui a funzionare regolarmente.

Il meccanismo si basa sulla ridondanza e sulla virtualizzazione degli indirizzi. Gli SSD sono fabbricati con una percentuale di blocchi fisici extra non visibili al sistema operativo (pratica detta *Over-Provisioning*). Quando il controller dell'SSD rileva che un blocco fisico ha raggiunto il limite di riscritture o presenta errori hardware, lo marca internamente come *Bad Block* (inutilizzabile). Da quel momento, il controller aggiornerà la tabella di mapping logico-fisico per reindirizzare le successive scritture su uno dei blocchi fisici di scorta, garantendo la totale trasparenza dell'operazione al sistema operativo e preservando l'integrità dei dati.

Domanda 3 (I/O Sincrono, Asincrono e Bloccante)

A) Differenze tra tipi di I/O e Sinonimi:

La differenza principale risiede in quando avviene il trasferimento effettivo dei dati e in quando il flusso di controllo ritorna al programma chiamante:

- **I/O Sincrono:** La system call ritorna il controllo al processo **solo dopo** che il trasferimento dei dati tra il dispositivo e il buffer utente è stato completato con successo (o è fallito definitivamente).
- **I/O Asincrono:** La system call ritorna **immediatamente**. Il trasferimento dei dati avviene in background (es. tramite DMA) parallelamente all'esecuzione del processo. Il processo verrà notificato in un secondo momento (tramite segnali o callback) ad operazione ultimata.

Bloccante/Non-Bloccante NON sono sinonimi di Sincrono/Asincrono.

Il concetto Bloccante/Non-Bloccante è un sottoinsieme dell'I/O Sincrono:

- **Bloccante:** Il thread viene rimosso dalla coda di ready e messo in stato di Wait/Sleep finché i dati non sono pronti.
- **Non-Bloccante:** La chiamata ritorna immediatamente *senza* bloccare il processo. Se i dati sono già pronti nel buffer del kernel, vengono copiati; se non c'è nulla, la chiamata ritorna un codice d'errore (es. **EAGAIN**) informando l'utente che dovrebbe riprovare più tardi. **Differenza chiave con l'Asincrono:** l'I/O non-bloccante non delega al kernel un trasferimento in background, si limita a un "prendi quello che c'è ora o lascia".

B) Polling vs Interrupts:

- **I/O Sincrono:** Può essere implementato in **entrambi** i modi. Può usare il Polling (*Programmed I/O*), dove la CPU cicla a vuoto verificando lo stato del registro del dispositivo, oppure essere *Interrupt-driven*, dove il processo si addormenta e viene risvegliato dall'interrupt di fine operazione.
- **I/O Asincrono:** A livello hardware/kernel, è **necessario** l'uso degli **interrupt** (spesso accoppiati al DMA). Poiché la CPU sta eseguendo il codice del processo utente sovrapponendolo all'I/O, non vi è alcun loop di polling attivo a livello di sistema operativo. È il dispositivo hardware che deve obbligatoriamente sollevare un'eccezione (interrupt) per informare il kernel che il buffer in background è stato riempito.

C) Benefici dell'I/O Asincrono e Dipendenze Dati:

- **Beneficio:** L'I/O asincrono permette la massima **sovrapposizione (overlap)** tra il tempo di calcolo della CPU (*CPU burst*) e il tempo di trasferimento del dispositivo (*I/O burst*). Questo massimizza il throughput del processo, che può fare calcoli complessi o gestire una UI mentre il disco legge i dati.
- **Dipendenze sui dati: SÌ**, è assolutamente possibile scrivere istruzioni che dipendono dai dati letti asincronamente, ma **non possono essere poste in sequenza lineare** subito dopo la system call di lancio. Devono essere ingabbiate all'interno di una routine specifica (un *Signal Handler* o una *Callback function*) che il sistema operativo invocherà autonomamente e asincronamente *solo nel momento in cui i dati saranno effettivamente validi e presenti nel buffer*. Se lette sequenzialmente prima dell'interrupt, si leggerebbe memoria "sporca" o non inizializzata.

Domanda 4 (System Call `sys_read` e Memoria)

A) Chiamata diretta da processo user:

NO, non è possibile chiamare `sys_read` direttamente.

Motivazione: La funzione `sys_read` appartiene al codice sorgente del kernel e risiede in segmenti di memoria protetti (`kseg0`). Un processo utente (come `palin.c`) viene eseguito in *User Mode* (`kuseg`) e non ha i privilegi hardware per effettuare salti (*jump/call*) verso indirizzi di memoria del kernel. L'unico modo in cui un processo utente può innescare l'esecuzione di `sys_read` è invocare il wrapper di libreria C `read()`, il quale solleva un'eccezione software (es. istruzione `syscall` in MIPS). Questa trap salva lo stato della CPU nel *trapframe*, commuta i privilegi in *Kernel Mode* e demanda al dispatcher del sistema operativo l'invocazione di `sys_read`.

B) Accesso scorretto di `kgets` e correzione:

Spiegazione del problema: La funzione esegue un cast brutale di `buf` (che è un puntatore non attendibile fornito dallo spazio utente) a `(char *)` e lo passa a `kgets`. `kgets` scriverà i dati provenienti dalla tastiera direttamente a quell'indirizzo. Se un processo malevolo passasse a `sys_read` un indirizzo appartenente allo spazio di memoria del kernel, il kernel finirebbe per sovrascrivere se stesso, causando un *Kernel Panic* o una violazione di sicurezza critica. Se passasse un indirizzo utente invalido (es. `NULL`), il kernel andrebbe in eccezione fatale.

Correzione proposta: Non bisogna mai far scrivere/leggere funzioni interne direttamente su puntatori `userptr_t`. Occorre:

1. Allocare un buffer temporaneo nello spazio del kernel (es. tramite `kmalloc` o un array locale se di piccole dimensioni).
2. Effettuare la lettura da standard input (es. con `kgets` o analoghi) all'interno del buffer del kernel.
3. Utilizzare la funzione sicura `copyout()` (o una struttura `uio` associata a `uiomove`) per trasferire i dati dal buffer del kernel al puntatore utente `buf`. `copyout` verificherà autonomamente la validità dell'indirizzo utente gestendo eventuali *page fault* in sicurezza.

C) Valore di ritorno e fluttuazioni rispetto a `size`:

- **Perché questa funzione ritorna `size`?** Questa realizzazione "ingenua" assume che `kgets` riempia esattamente tutto il buffer richiesto dall'utente, omettendo di controllare quanti caratteri siano stati effettivamente digitati.
- **Può essere MINORE di `size`? SÌ.**
In una realizzazione conforme allo standard POSIX (es. tramite le strutture `vnode` e `uio`), una lettura può ritornare meno byte di quelli richiesti (*Short Read*). Questo accade frequentemente se si raggiunge la fine del file (*EOF*), se si legge da un terminale interattivo (ritorna alla pressione dell'Invio), o se il file è un pipe/socket non completamente pieno.
- **Può essere MAGGIORE di `size`? MAI.**
Un valore di ritorno maggiore di `size` implicherebbe che il sistema operativo ha scritto in memoria più byte di quanti ne misuri il buffer allocato dall'utente. Questo provocherebbe inevitabilmente un *Buffer Overflow* nello spazio di indirizzamento dell'utente, corrompendone i dati o lo stack.

Domanda 5 (Sincronizzazione Multi-core e Semafori)

A) Inadeguatezza della disabilitazione interrupt nei sistemi Multi-core:

Nei sistemi a singola CPU, disabilitare gli interrupt è sufficiente per garantire la mutua esclusione,

poiché si inibisce l'orologio hardware e si blocca la preemption (lo scheduler non può subentrare e cambiare contesto). In un contesto multi-core, invece, l'operazione di disabilitazione degli interrupt è strettamente **locale al singolo processore**. Se un thread in esecuzione sulla CPU 1 disabilita i propri interrupt, un altro thread in esecuzione concorrente sulla CPU 2 continuerà a lavorare regolarmente, potendo accedere parallelamente alla medesima risorsa condivisa violando la mutua esclusione.

B) Analisi del codice delle funzioni P e V del semaforo:

1. **Ruolo dello spinlock:** Lo spinlock è necessario per proteggere la sezione critica interna del semaforo stesso. Garantisce che il controllo e l'aggiornamento della variabile `sem_count` e l'accesso alla coda del `wait_channel` avvengano in modo strettamente atomico.
2. **Uso del ciclo while (Semantica Mesa):** Il risveglio non cede direttamente il lock al thread addormentato. Quando `wchan_wakeone` viene chiamata, il thread sospeso viene semplicemente spostato nella coda di *Ready*. Prima che quest'ultimo venga effettivamente schedato e riacquisisca lo spinlock per uscire dalla `sleep`, un terzo thread (appena arrivato) potrebbe prendere la CPU, acquisire lo spinlock, superare il controllo, decrementare il contatore e rilasciare il lock. Quando il thread inizialmente risvegliato ottiene finalmente lo spinlock, troverebbe la risorsa già "rubata" (`sem_count` tornato a 0). Il `while` è vitale per forzare la rivalutazione della condizione.
3. **Parametro spinlock in wchan_sleep:** Il passaggio del riferimento al lock è obbligatorio per prevenire scenari di **deadlock**. La funzione `wchan_sleep` deve occuparsi di rilasciare lo spinlock *esattamente nello stesso istante* (in modo atomico) in cui il thread viene sospeso. Se il thread andasse in stato di sleep mantenendo il possesso dello spinlock, nessun altro thread potrebbe mai eseguire la funzione V per risvegliarlo, in quanto rimarrebbe bloccato all'infinito sulla `spinlock_acquire` iniziale.
4. **Wakeone multipli: NO.** La chiamata `wchan_wakeone` sveglia esplicitamente un singolo thread, se presente in coda. Per i risvegli multipli o broadcast si utilizza l'apposita funzione `wchan_wakeall`.

C) Differenza tra Condition Variable e Wait Channel:

Occorre distinguere tra l'astrazione concettuale e il meccanismo di basso livello in OS161:

- Il **Wait Channel (wchan)** è una primitiva di livello bassissimo del kernel OS161. Rappresenta puramente la struttura dati (la coda di thread) usata dallo scheduler per mantenere i processi in stato di "sonno". Da solo, non ha alcuna semantica logica di sincronizzazione, ma funge da "mattoncino" base per costruire gli altri costrutti.
- La **Condition Variable (CV)** è invece una struttura di sincronizzazione di alto livello logico. Implementa il pattern architetturale "Monitor" in combinazione con un Lock esterno. Sebbene la CV nasconda al suo interno un `wchan` per gestire l'addormentamento fisico dei thread, vi aggiunge sopra una semantica rigorosa legata a predicati e condizioni applicative, esponendo un'interfaccia sicura (`cv_wait`, `cv_signal`, `cv_broadcast`).